System Hardware

SOEN228

Lab Report #2

Student Name	Student ID	Lab section	Lecture section
Michaël Gugliandolo	40213419	C-Z (lab D)	U
Jean-Nicolas Sabatini-Ouellet	40207926	B	U

Department of Computer Science & Software Engineering

Concordia University

Winter 2022

Objectives **2**

Introduction **2**

Experimental Results and Questions **3**

Discussion **6**

Conclusions **7**

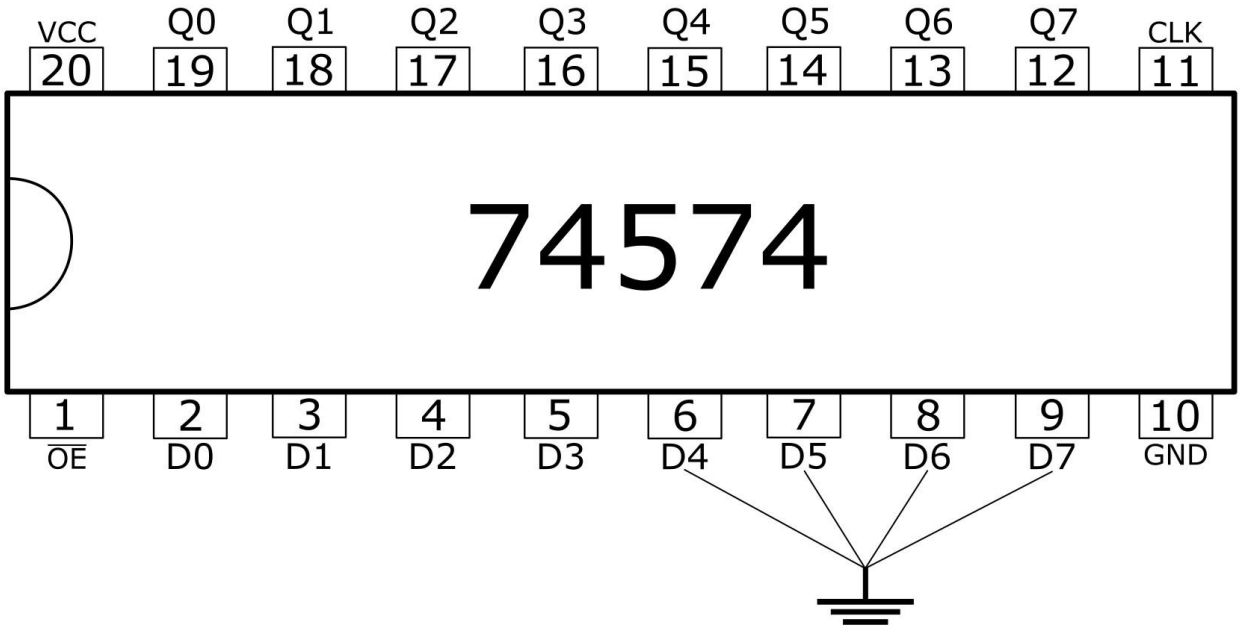
1. Objectives

This lab consists of creating our first program counter and 4-bit bus. We will use multiple registers and a 4-bit adder. Our arithmetic unit, program counter and bus will be the building blocks for more complicated circuits.

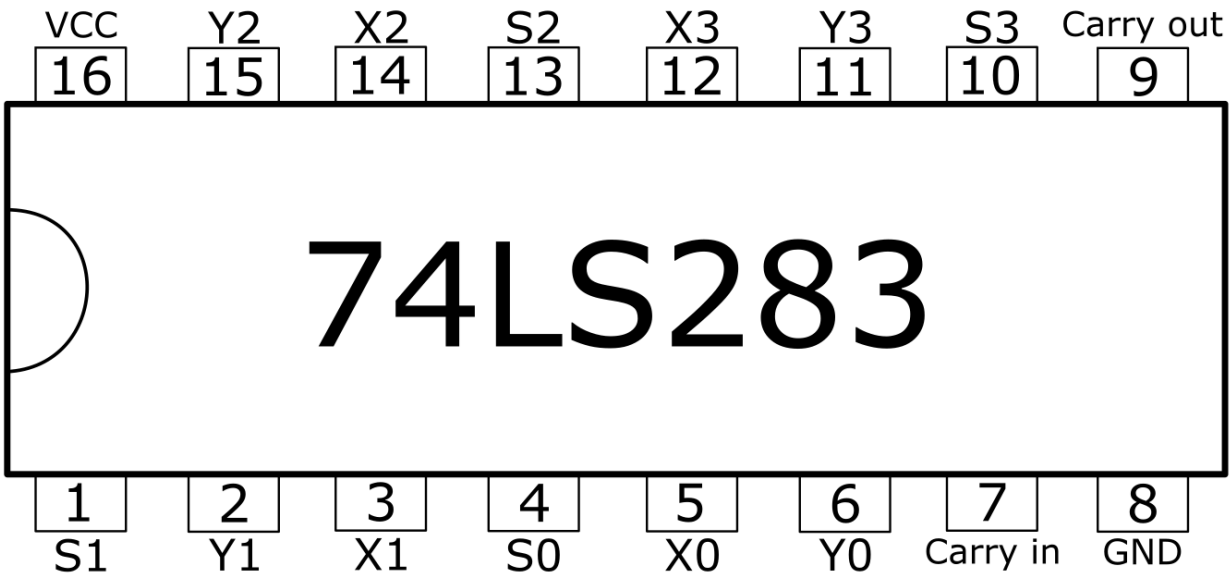
2. Introduction

In this lab, we will construct a bus, which is like a highway of information linking components together. We will also build a program counter, which is a register with the address of the current instruction, and it increments periodically, from 0000 to 1111, then back to 0000, etc.

We will use 74HC574 8-bit registers as 4-bit registers by grounding the last 4 input bits. A register has internal flip-flops to hold binary values in memory. In addition, it has an output enable (OE) bit to control when the register can output its value; it can only output when its OE has a value of 0. Coupled with a timing signal generator (TSG), we can control which register outputs to the bus to avoid bus conflicts (when multiple devices output at the same time).

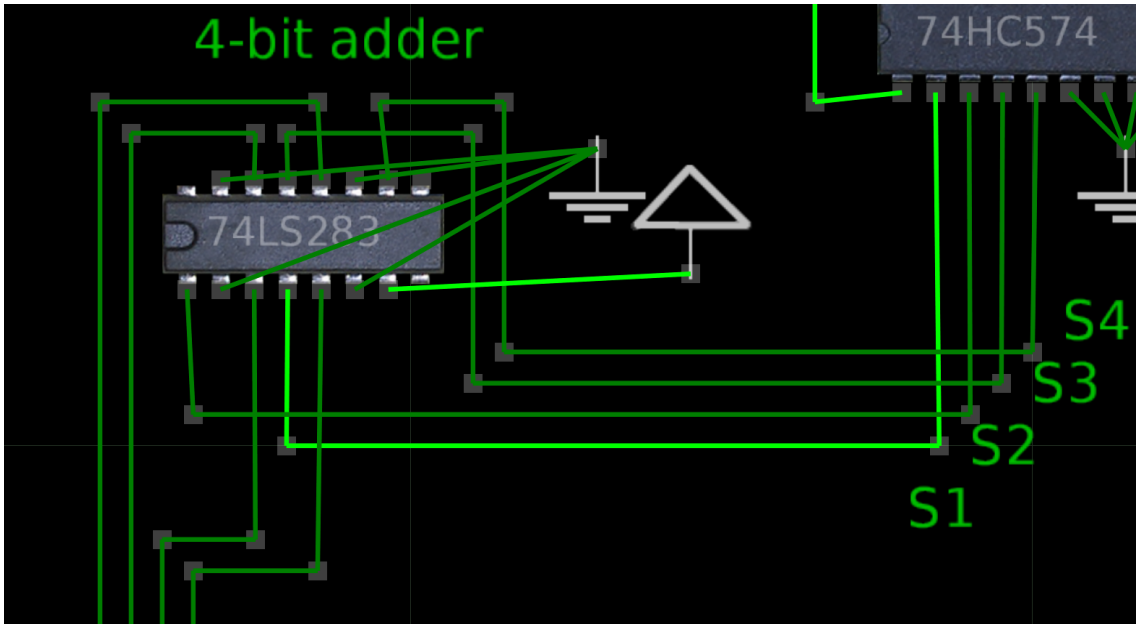


Furthermore, a 4-bit adder takes two 4-bit numbers and a carry-in as inputs. It adds them together and outputs a 4-bit sum and a carry-out. The 74LS283 chip will be used, where X and Y are the two input numbers and S is the sum:



3. Experimental Results and Questions

The first input number of our adder is connected to the bus, and the second input number pins are grounded. We will connect the carry-in to logic 1, so the adder will always take a number from the bus and increment by 1.



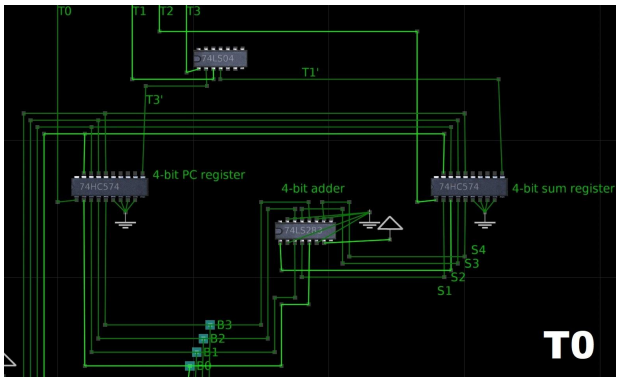
To control our program counter (PC), we will use the first four output bits of our TSG.

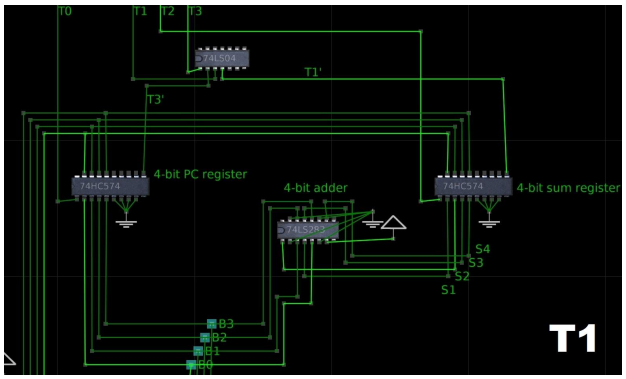
T0: Turn off the OE of the PC register so it outputs its value to the bus. The adder will take this value and add 1.

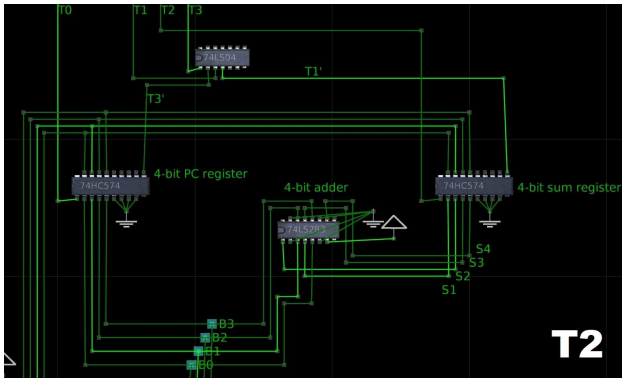
T1: It's connected to the clock of the sum register, so the register reads the number from the adder. Since our computer is positive-edge triggered, we have to invert the signal from the TSG.

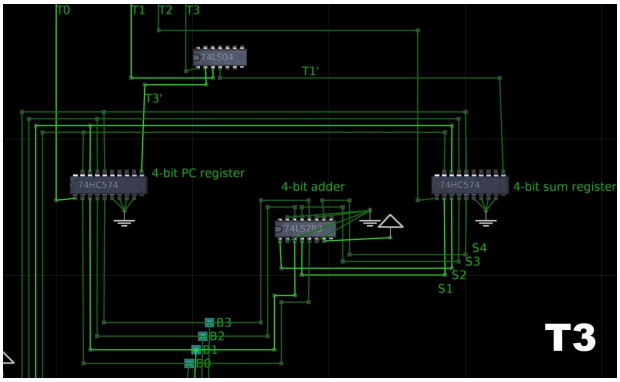
T2: Turn off the OE of the sum register so it outputs its value to the bus.

T3: It's connected to the clock of the PC register, so the register reads the number from the sum register (that's on the bus). Since our computer is positive-edge triggered, we have to invert the signal from the TSG.

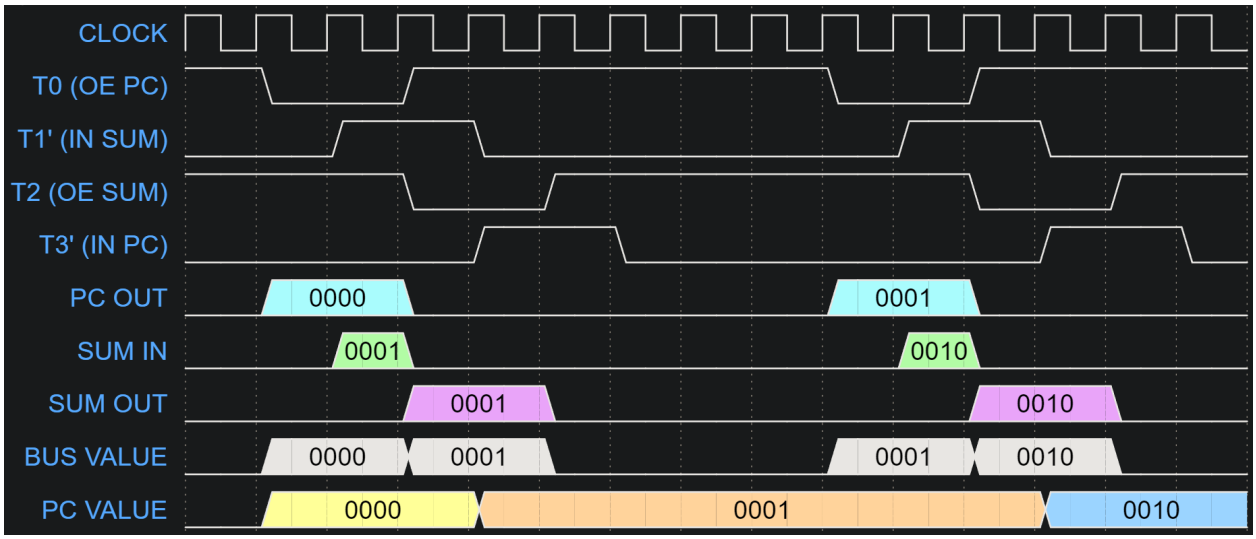








Here is our timing diagram:



CLOCK	5Hz clock of TSG
T0 (OE PC)	Output bit 1 of TSG, linked to OE of PC register (when 0, outputs value to bus)
T1' (IN SUM)	Inverted output bit 2 of TSG, linked to clock of sum register (when 1, reads value from adder)
T2 (OE SUM)	Output bit 3 of TSG, linked to OE of sum register (when 0, outputs value to bus)
T3' (IN PC)	Inverted output bit 4 of TSG, linked to clock of PC register (when 1, reads value from bus)
PC OUT	Output of the PC register to the bus (read and incremented by the adder)
SUM IN	Input of the sum register from the adder (positive-edge triggered)
SUM OUT	Output of the sum register to the bus
BUS VALUE	Current value on the bus
PC VALUE	Value stored in the PC register

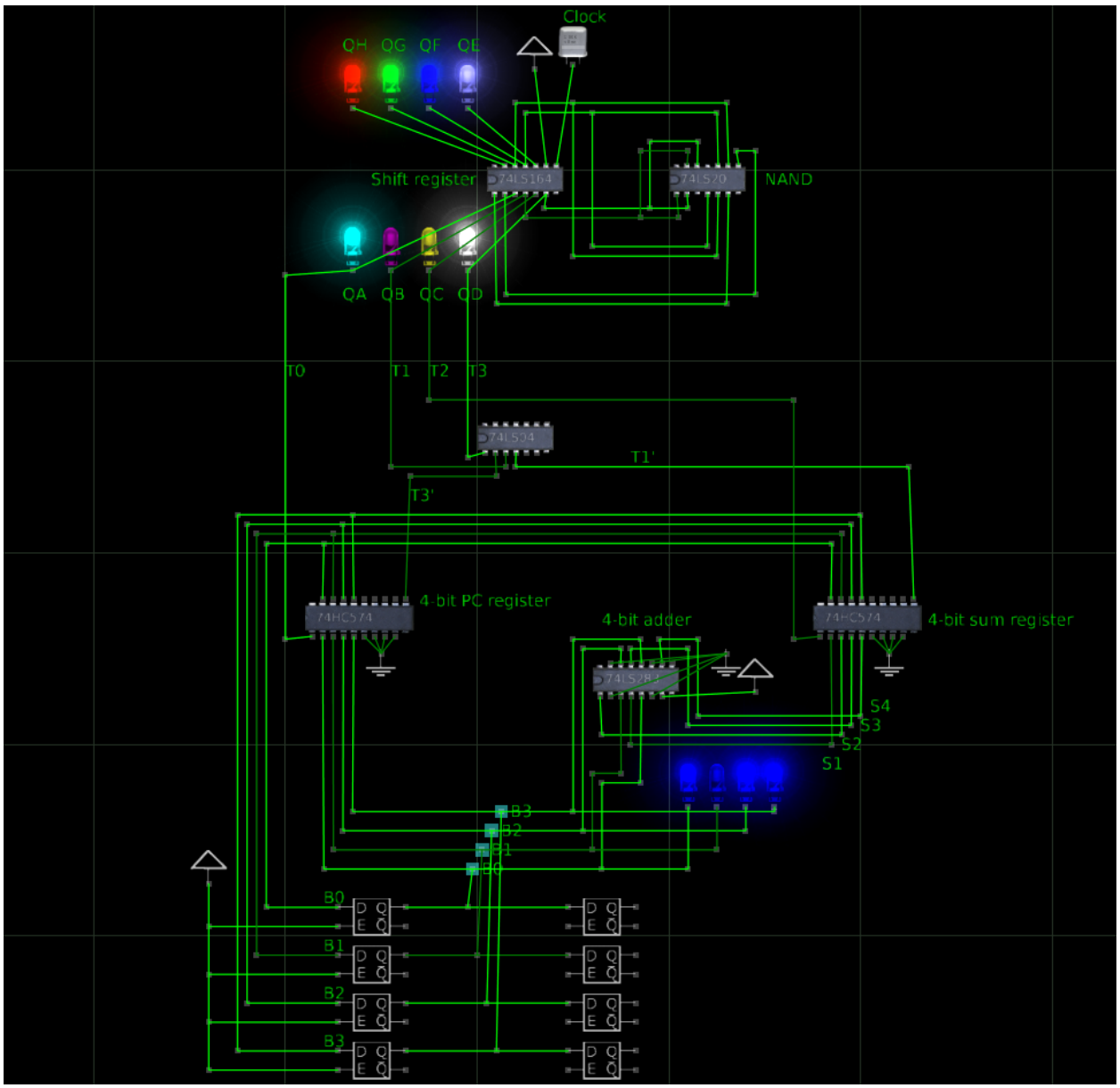
1. Why does Atanua require that the unused inputs for the 74HC574 be grounded?

In order for the Atanua simulation to work, all of the input pins have to be connected, and since these bits are unused, we can ground them to set their value to logic 0. Registers store memory; thus, an 8-bit register must have exactly 8 inputs. Furthermore, logic 0 doesn't mean 'no signal', it means 'low signal'. If some of the inputs are not connected, the chip will not function since there is no signal passing through. For example, if 6 bits were needed for a register, it would be much more efficient to have a 6-bit register instead of an 8-bit with 2 grounded inputs.

2. What would happen if the circuit did not have the sum register and the outputs of the adder were connected directly to the bus?

There wouldn't be any buffer. When the PC register stops outputting to read the new value, the new value will immediately disappear from the bus. An adder is a combinational circuit, meaning that it doesn't have memory. When the PC register stops outputting, the incremented value will disappear with it.

4. Discussion



The program counter doesn't increment as quickly as our 5 Hz clock, since it takes multiple signals from the TSG for a complete incrementation by 1. In fact, according to our timing diagram, it takes 4 clock cycles to increment. Also, we can see how easy it is to have bus conflicts since by adding more and more components, the bus will have more connections and more potential for bus conflicts. The importance of keeping the bus clutter-free is therefore apparent.

It's also clear how a faster clock can increase the speed of the computer. A faster clock will save fractions of seconds every step, which quickly adds up to seconds for longer instructions.

5. Conclusions

In this lab, we learnt about the program counter, the bus and the arithmetic unit (the 4-bit adder). We used 8-bit registers and converted them into 4-bit, and we used our TSG to control which component has access to the bus to avoid bus conflicts. Combining all of this, we build a functional program counter that increments from 0000 to 1111 and starts over at 0000. We finally represented everything in a timing diagram.